**TDT4237 2024 Sensor guide**

**The letter grade follows the NTNU grading scale using percentage points (**Grading scale using percentage points - Knowledge base - NTNU**)**

**A:** 89–100 points

**B:** 77–88 points

**C:** 65–76 points

**D:** 53–64 points

**E:** 41–52 points

**F:** 0–40 points

**Answers to questions and corresponding grading guidelines**

**Q2: Strategy to mitigate the security compromise impact (4 points)**

Suppose your system takes users' input and can be exposed to injection attacks. List and explain at least four strategies to mitigate the impact of injection attack compromises. (4 points)

[redacted]

**Q3:** Question undefinedSecurity and Integrity of OTP (4 points)

1. Explain encryption and decryption algorithm of One Time Pad (OTP). (1 point)

2. Explain why it is insecure to use the same key to encrypt two or several messages using OTP. (2 points)

3. Explain why OTP cannot guarantee integrity (1 point)

[redacted]

[redacted]

**Q4:** AI and security (4 points)

What is the difference between malicious abuse of AI and malicious use of AI? Give examples of both.

[redacted]

**Q5:** Privacy-related question (4 points)

How can a controller demonstrate data protection? Give at least 5 examples.

[redacted]

[redacted]

Q6: Code to setup password policy (4 points)

Suppose the password policy of a system is as follows.

- The password is between 8-10 characters long
- The password contains characters from 3 of the following 4 categories:
    - standard uppercase characters (A - Z)
    - standard lowercase characters (a - z)
    - numbers (0 - 9)
    - symbols: only from among ! % - _ + = [ ] { } : , . ? < > ( ) ;
- The password **does not** contain information identical to user's first and last name
- The password **does not** contain common passwords
- **Spaces and the letters "æ", "ø" and "å" are not accepted**

[redacted]

Q7: Circuit breaker pattern (4 points)

What is the purpose of the circuit breaker pattern in the context of microservice architecture security?

[redacted]

[redacted]

Q8: Question undefined Authorization (4 points)

1) In the Bell-LaPadula model, what does the * property mean?

2) What about STRONG * ?

[redacted]

Q9: Concept drift (4 points)

What is the concept drift challenge within AI? Give an example.

[redacted]

Q10: Penetration Testing (4 points)

What are the pros and cons of penetration testing tools?

[redacted]

Q11: Social Engineering (4 points)

Mention principles of persuasion that can be used for social engineering attacks.

[redacted]

Security logging vulnerability fixing (4 points)

production.py

```
from app.settings.common import *
from decouple import config, Csv


# Email admins and managers
# https://docs.djangoproject.com/en/2.1/howto/deployment/checklist/#admins-and-managers
ADMINS = config('ADMINS', default=[('Admin', 'admin@forum.securecodewarrior.com')], cast=Csv())
MANAGERS = config('MANAGERS', default=[('moderator', 'moderator@forum.securecodewarrior.com')], cast=Csv())


# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = False

# https://docs.djangoproject.com/en/2.1/ref/settings/#allowed-hosts
ALLOWED_HOSTS = ['forum.securecodewarrior.com', ]


# SSL/HTTPS
# https://docs.djangoproject.com/en/2.1/topics/security/#ssl-https

SECURE_SSL_REDIRECT = True

SESSION_COOKIE_SECURE = True

# Cross Site Request Forgery
CSRF_COOKIE_SECURE = True
CSRF_TRUSTED_ORIGINS = config('CSRF_TRUSTED_ORIGINS', default=[], cast=Csv())

SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')
```

```
30
31
32 # https://docs.djangoproject.com/en/2.1/topics/security/
33 # HSTS
34 SECURE_HSTS_SECONDS = 5 * 60
35 SECURE_HSTS_PRELOAD = True
36 SECURE_HSTS_INCLUDE_SUBDOMAINS = True
37
38 # https://docs.djangoproject.com/en/2.1/ref/clickjacking/
39 X_FRAME_OPTIONS = 'DENY'
40
41 SECURE_CONTENT_TYPE_NOSNIFF = True
42
43 SECURE_BROWSER_XSS_FILTER = True
44
45 # Sessions
46 # https://docs.djangoproject.com/en/2.1/ref/settings/#sessions
47 SESSION_COOKIE_AGE = 24 * 60 * 60  # 24 hours, in seconds
48 SESSION_EXPIRE_AT_BROWSER_CLOSE = True
49
50
51 # Database
52 # https://docs.djangoproject.com/en/2.1/ref/settings/#databases
53
54 DATABASES = {
55     'default': {
56         'ENGINE': 'django.db.backends.postgresql',
57         'NAME': config('DATABASE_NAME'),
58         'USER': config('DATABASE_USER'),
59         'PASSWORD': config('DATABASE_PASSWORD'),
60         'HOST': config('DATABASE_HOST'),
61         'PORT': config('DATABASE_PORT'),
62     }
63 }
64
65
66 # Email configuration
67 # https://docs.djangoproject.com/en/2.1/ref/settings/#default-from-email
68
69 DEFAULT_FROM_EMAIL = 'support@forum.securecodewarrior.com'
70 SERVER_EMAIL = 'admin@forum.securecodewarrior.com'
71
72
73 # https://docs.djangoproject.com/en/2.1/ref/settings/#email-backend
74 EMAIL_BACKEND = config('EMAIL_BACKEND')
75 EMAIL_HOST = config('EMAIL_HOST')
76 EMAIL_PORT = config('EMAIL_PORT', cast=int)
77 EMAIL_USE_TLS = config('EMAIL_USE_TLS', cast=bool)
78 EMAIL_HOST_USER = config('EMAIL_HOST_USER')
79 EMAIL_HOST_PASSWORD = config('EMAIL_HOST_PASSWORD')
```

```
80
81
82 # https://docs.djangoproject.com/en/2.1/ref/settings/#file-upload-permissions
83 FILE_UPLOAD_PERMISSIONS = config('FILE_UPLOAD_PERMISSIONS',
default=0o640, cast=oct)
```

Code snippet of common.py

```
1 # Logging
2 # https://docs.djangoproject.com/en/2.1/topics/logging/#configuring-logging
3
4 # Disable Django's logging setup
5 LOGGING_CONFIG = None
6
7 LOGLEVEL = config('LOGLEVEL', default='INFO')
8
9 # https://docs.djangoproject.com/en/2.1/topics/logging/#custom-logging-configuration
10 logging.config.dictConfig({
11     'version': 1,
12     'disable_existing_loggers': False,
13     'formatters': {
14         'default': {
15             # exact format is not important, this is the minimum information
16             'format': '%(asctime)s %(name)-12s %(levelname)-8s %(message)s',
17         },
18         'django.server': DEFAULT_LOGGING['formatters']['django.server'],
19     },
20     'handlers': {
21         # console logs to stderr
22         'console': {
23             'class': 'logging.StreamHandler',
24             'formatter': 'default',
25         },
26         'file': {
27             'level': 'WARNING',
28             'class': 'logging.FileHandler',
29             'filename': os.path.join(BASE_DIR, 'debug.log'),
30         },
31         'django.server': DEFAULT_LOGGING['handlers']['django.server'],
32     },
33     'loggers': {
34         # default for all undefined Python modules
35         '': {
36           'level': LOGLEVEL,
37             'handlers': ['console', 'file'],
38         },
39         # Prevent noisy modules from logging
40         'noisy_module': {
41             'level': 'ERROR',
42             'handlers': ['console'],
43             'propagate': False,
44         },
```

```
        # Default runserver request logging
        'django.server': DEFAULT_LOGGING['loggers']['django.server'],
    },
})
```

The code snippet common.py has vulnerabilities related to security logging and monitoring. The code in production.py provides some background information.

- Explain which lines in common.py are vulnerable and why they are vulnerable. (2 points)

- Explain how to fix the vulnerabilities in the common.py. It is necessary to provide code to show how to fix it. (2 points) (Note: Syntax errors are allowed, especially if you explain the code.)

[redacted]

[redacted]

[redacted]

[redacted]

**Q13:** Supply chain security (4 points)

What are the four steps of software supply chain attacks and what are the corresponding countermeasure strategies? (4 points)

[redacted]

[redacted]

Q14: Microservice architecture attack area (4 points)

What are potential attack areas of microservices deployed on a cloud? (4 points)

[redacted]

## 14 Microservice architecture attack area (4 points)

What are potential attack areas of microservices deployed on a cloud? (4 points)

**Fill in your answer here**

Format

Words: 0

## 15 Privacy motivation (1 Point)

What is the biggest motivation for software companies to work with privacy?

**Select one alternative:**

- Big legal fines
- This is what management cares about
- This is what developers care about
- The respect of their customers
- Catching criminals

[redacted]

## 16 Cryptography keys (1 point)

Which of the following methods is NOT a recommended approach for generating cryptographic keys?

**Select one alternative:**

- Collecting entropy from user-generated input, such as mouse movements or keyboard strokes. [redacted]
- Using a hardware random number generator [redacted]
- Employing a software-based secure pseudo-random number generator with unique seeds [redacted]
- Reusing a previously generated key for a new encryption task [redacted]
- Deriving keys from a passphrase using a key derivation function [redacted]

## 17 Threat modeling (1 point)

What is the best way of performing threat modeling?

**Select one alternative:**

- You should create your own threat modeling technique that is tailored for the j[redacted]
- Attack trees were the first and is still the most recognized way of modeling threat[redacted]
- Misuse case diagrams were invented at NTNU and considered to be the most useful way.[redacted]
- It is better to create multiple threat modeling representations because there is no s[redacted] ideal view
- DFD is the most widely used threat modeling technique and should therefore be used[redacted]

## 18 Injection vulnerability in the code (1 point)

cart.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title> User Cart </title>
</head>
<body>
     {% for item in items %}

          Product Name: {{ item.product_name }}<br>
          Product Id : {{ item.product_id }}<br>
          Transaction Id : {{ item.transaction }}<br>
          Price : {{ item.price }}<br>

     {% endfor %}
<br><br><b><a href="{% url 'shops:logout' %}"> logout</a></b>
</body>
</html>
```

models.py

```
from __future__ import unicode_literals

from django.db import models
from django.contrib.auth.models import User


# Model for shopping order transactions.
class ShoppingTransaction(models.Model):
    order_id = models.CharField(max_length=20)
    date = models.DateTimeField()
    total_amount = models.DecimalField(max_digits=7, decimal_places=2)
    user = models.ForeignKey(User)

    def __str__(self):
        return self.order_id


# Model for products purchased by a customer.
class TransactionDetail(models.Model):
    product_id = models.CharField(max_length=20)
    product_name = models.CharField(max_length=40)
    transaction = models.ForeignKey(ShoppingTransaction)
    price = models.DecimalField(max_digits=7, decimal_places=2)
```

views.py

```
from django.shortcuts import render
from django.http import HttpResponseRedirect
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
from django.core.urlresolvers import reverse
from django.contrib import messages

from shops.models import TransactionDetail, ShoppingTransaction
from shops.forms import LoginForm


# User Login
def user_login(request):
    # Checking the request method
    if request.method == 'POST':
        # Create a form instance and populate it with data from the request:
        form = LoginForm(request.POST)
        # Checking if all the form fields value meet the set criteria
        if form.is_valid():
            # Fetching the username and passwords from POST methods
            user_name = form.cleaned_data['username']
            pass_word = form.cleaned_data['password']
            # Authenticating the user
            user = authenticate(username=user_name, password=pass_word)
            # Checking if the user is successfully authenticated
            if user is not None:
                # Login the user and creating a user session
                if user.is_active:
                    login(request, user)
                    return HttpResponseRedirect(reverse('shops:order'))
                else:
                    messages.error(request, 'User is disabled.')
            else:
                form = LoginForm()
                # Flashing a message on incorrect login credentials
                messages.error(request, 'Incorrect Login Details .. Please try again')
```

```
37     else:
38         # Creating a form instance
39         form = LoginForm()
40
41     return render(request, 'shops/login.html', {'form': form})
42
43
44 # Fetching all the orders for that user
45 @login_required(login_url='/shops/login/')
46 def user_order_view(request):
47     data_set = ShoppingTransaction.objects.filter(user=request.user)
48     return render(request, 'shops/order.html', {'orders': data_set})
49
50
51 # Order details page
52 @login_required(login_url='/shops/login/')
53 def user_cart_view(request, transaction_id):
54     data_set = TransactionDetail.objects.filter(transaction=transaction_id)
55     return render(request, 'shops/cart.html', {'items': data_set})
56
57
58 # Logout Page
59 @login_required(login_url='/shops/login/')
60 def user_logout(request):
61    logout(request)
62    return render(request, 'shops/logout.html', {})
```

In the above code snippets, which lines are vulnerable?
**Select one alternative:**

- views.py: 54-55
- models.py: 11-12
- models.py: 22-23
- cart.html: 10-11

[redacted]

## 19 CIA triad (1 point)

Which of the following principles is part of the CIA triad?
**Select one alternative:**

- Auditability: Enables monitoring and recording of system activities for security analysis [redacted]
- Accountability: refers to the principle that an individual is entrusted to safeguard and control equipment, keying material, and information.
- Attacks: involve direct actions against a system or network.
- Availability: Ensures that authorized users can access data when needed.
- Authentication: Verifies the identity of users or processes.

[redacted]

## 20 Authorization vulnerability in the code (1 point)

models.py

```
from __future__ import unicode_literals

from django.db import models
from django.contrib.auth.models import User


# Model for team participating in competition.
class Team(models.Model):
   name = models.CharField(max_length=15)

   def __str__(self):
       return self.name


# Model for gamer profiles.
class GamerProfile(models.Model):
   alias_name = models.CharField(max_length=40)
   game_name = models.CharField(max_length=30)
   score = models.IntegerField()
   team = models.ForeignKey(Team)
   user = models.ForeignKey(User)

   def __str__(self):
       return self.alias_name
```

views.py

```
from django.shortcuts import render
from django.contrib.auth import authenticate, login, logout
from django.core.urlresolvers import reverse
from django.http import HttpResponseRedirect, HttpResponse
from django.contrib import messages
from django.contrib.auth import decorators
from django.shortcuts import get_object_or_404

from games.models import GamerProfile, Team
from games.forms import LoginForm


# User login process
def user_login(request):
   # Checking the request method
   if request.method == 'POST':
       # Create a form instance and populate it with data from the request
       form = LoginForm(request.POST)
       if form.is_valid():
           # Fetching the username and passwords from POST methods
           user_name = form.cleaned_data['username']
           pass_word = form.cleaned_data['password']
           # Authenticating the user
           user = authenticate(username=user_name, password=pass_word)
           # Checking if the user is successfully authenticated
           if user is not None:
               # Login the user and creating a user session
               if user.is_active:
                   login(request, user)
                   return HttpResponseRedirect(reverse('games:dashboard'))
               else:
                   messages(request, 'User is disabled.')
           else:
               form = LoginForm()
               messages.error(request, 'Incorrect Login Details. Please try ag ain')
   else:
       # Instantiating empty form
       form = LoginForm()

   return render(request, 'games/login.html', {'form': form})


# User gaming dashboard
@decorators.login_required(login_url='/games/login/')
def dashboard(request):
   team = get_object_or_404(Team, user=request.user)
   team_gamers = GamerProfile.objects.filter(team=team.team)
   return render(request, 'games/dashboard.html', {'team_gamers': team_gamers, })


# User Team members
@decorators.login_required(login_url='/games/login/')
def gamer_profile(request, gamer_id):
   gamer_details = get_object_or_404(GamerProfile, pk=gamer_id)
   return render(request, 'games/gamer_details.html', {'gamer': gamer_details, })
```

```


# User logout
@decorators.login_required(login_url='/games/login/')
def log_out(request):
    logout(request)
    return render(request, 'games/logout.html', {})
```

settings.py

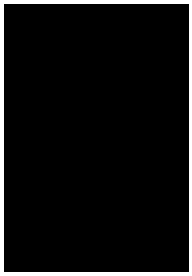
```
# -*- coding: utf-8 -*-

#
# settings file for production environment
#
# This settings provides the MINIMUM level of security. Additional
# settings may be used to hardening the system (not added here because of
# potential compatibility issues with the software), like, for example:
#
#  - SECURE_PROXY_SSL_HEADER
#  - SECURE_HSTS_SECONDS
#  - SECURE_HSTS_INCLUDE_SUBDOMAINS
#  - SECURE_SSL_REDIRECT
#  - SECURE_SSL_HOST
#


from __future__ import unicode_literals

import os

from django.core.exceptions import ImproperlyConfigured

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'games.apps.GamesConfig',
]

ROOT_URLCONF = 'website.urls'

WSGI_APPLICATION = 'website.wsgi.application'

DEBUG = False

ALLOWED_HOSTS = [
    'randomapp.securecodewarrior.com'
]

CSRF_COOKIE_SECURE = True
SESSION_COOKIE_SECURE = True

try:
    SECRET_KEY = os.environ['DJANGO__SECRET_KEY']

    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.postgresql',
            'NAME': os.environ['DJANGO__DB_NAME'],
            'USER': os.environ['DJANGO__DB_USER'],
            'PASSWORD': os.environ['DJANGO__DB_PASSWORD'],
            'HOST': os.environ['DJANGO__DB_HOST'],
            'PORT': os.environ['DJANGO__DB_PORT'],
        }
    }

except KeyError, ex:
    key = ex.args[0]
    raise ImproperlyConfigured("The environment variable {0} "
                               "was not found and is required".format(key))

MIDDLEWARE_CLASSES = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.auth.middleware.SessionAuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

TEMPLATES = [
```

```
78     {
79         'BACKEND': 'django.template.backends.django.DjangoTemplates',
80         'DIRS': [],
81         'APP_DIRS': True,
82         'OPTIONS': {
83             'context_processors': [
84                 'django.template.context_processors.debug',
85                 'django.template.context_processors.request',
86                 'django.contrib.auth.context_processors.auth',
87                 'django.contrib.messages.context_processors.messages',
88             ],
89         },
90     },
91 ]
92
93 AUTH_PASSWORD_VALIDATORS = [
94     {
95         'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
96     },
97     {
98         'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
99     },
100     {
101         'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
102     },
103     {
104         'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
105     },
106 ]
107
108 STATIC_URL = '/static/'
```

In the above code snippets, which lines of code are vulnerable?

**Select one alternative:**

- views.py:46-47
- settings.py:71-72
- views.py:54-54
- models.py: 20-21



## 21 Crypto vulnerability in the code (1 point)

```
from __future__ import unicode_literals

import hashlib
from datetime import date
from datetime import timedelta

from django.db import models
from django.conf import settings


class Payment(models.Model):
    """
    Represents a payment.

    It could be pending to be processed (`accepted` is None)
    or already processed (`accepted` is True or False).
    """
    description = models.CharField(max_length=50)
    payment_from = models.ForeignKey(settings.AUTH_USER_MODEL, related_name='+')
    payment_to = models.ForeignKey(settings.AUTH_USER_MODEL, related_name='+')
    amount = models.DecimalField(max_digits=9, decimal_places=2)
    accepted = models.NullBooleanField(null=True)


class InvalidSecurityCode(Exception):
    """The provided security code is not valid"""


class SecurityCodeManager(models.Manager):
    @staticmethod
    def encrypt_security_code(plaintext_security_code):
        """
        Encrypt the provided plain-text security code
        :param plaintext_security_code: plain-text security code
        :return: crypted security code
        """
        assert isinstance(plaintext_security_code, unicode)

        hash_inst = hashlib.md5(plaintext_security_code.encode('utf-8'))
        return hash_inst.hexdigest()

    def check_security_code(self, plaintext_security_code):
        """
        Verifies if the provided plain-text security code
        exists in the database, and isn't too old.

        Raises an exception if the code isn't valid.

        :param plaintext_security_code: the security code in plain text
                                        format (as entered by the user)
        :raise InvalidSecurityCode: if code is not valid
        """
        crypted_code = self.encrypt_security_code(plaintext_security_code)

        today = date.today()
        valid_from_date = today - timedelta(days=15)

        # control date and ignore time, we no need such precision
        qs = self.filter(created_at__date__gte=valid_from_date)
        qs = qs.filter(crypted_password=crypted_code)

        if not qs.exists():
            raise InvalidSecurityCode()

    def delete_old_security_codes(self):
        """
        Delete old security codes from the database.

        This should be called periodically to avoid having
        old codes in the database.
        """
        today = date.today()
        valid_from_date = today - timedelta(days=15)

        self.filter(created_at__date__lt=valid_from_date).delete()


class SecurityCode(models.Model):
    """
    Represents a security code to be entered by the user
    to prove the authenticity when processing a payment.
    """
```

```
83    created_at = models.DateTimeField(auto_now_add=True)
84    crypted_password = models.CharField(max_length=1000, db_index=True)
85
86    objects = SecurityCodeManager()
87
88    def set_security_code(self, plaintext_code):
89        """
90        Crypt and set the security code in this instance, based on the
91        provided plain-text security code.
92
93        Use of this method must call save() to update the database.
94        """
95
96        if len(set(list(plaintext_code.lower()))) < 10:
97            raise InvalidSecurityCode("The provided text is "
98                                      "not valid and can not be used as "
99                                      "a security code")
100
101        self.crypted_password = SecurityCodeManager.encrypt_security_code(
102            plaintext_code)
103
104    def __str__(self):
105        return "Security code {} ({})".format(self.id, self.created_at)
```

Which lines of the above code snippet are vulnerable?

**Select one alternative:**

- 83-84
- 72-75
- None of the above listed lines are vulnerable.
- 39-40

[redacted]

## 22 Security guiding principle (1 point)

Which security guiding principle is related to the blacklisting countermeasure?

**Select one alternative:**

- Promote privacy
- Keep it difficult
- Practice defense in depth
- Keep it simple
- Be reluctant to trust

[redacted]

## 23 OWASP vulnerability in the code (1 point)

```
index.html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Random App</title>
6 </head>
7 <body>
8 {# Message notifications #}
9    {% if messages %}
10       <ul class="messages">
11              {% for message in messages %}
12                 <li {% if message.tags %} class="{{ message.tags }}"{% endif %} >{{ message }}</li>
13              {% endfor %}
14       </ul>
15    {% endif %}
16    {# Message notifications end #}
17    <h4>Check host status</h4>
18         <form  method="POST" action="{% url 'host:index' %}">
19             {% csrf_token %}
20              {{ form }}
21          <br />
22             <input type="submit" value="submit" />
23         </form><br>
24    <hr>
25    <br>
26    {% if request.POST %}
27       <h4>{{ output }}</h4>
28    {% endif%}
29    <br>
30
31 </body>
32 </html>
```

```
forms.py
1 from django import forms
2
3
4 # Form architecture to find host status
5 class HostCheckForm(forms.Form):
6     ip = forms.CharField()
```

```
views.py
1 from django.shortcuts import render
2
3 from host.forms import HostCheckForm
4
5 from os import popen2
6
7
8 # View method to check host status
9 def index(request):
10     output = None
11     # Checking request method
12     if request.method == 'POST':
13         # Initialising form with POST request
14         form = HostCheckForm(request.POST)
15         # Validating form inputs
16         if form.is_valid():
17             cmd_string = 'ping -c 3 ' + form.cleaned_data['ip']
18             process_output = popen2(cmd_string, mode='r', bufsize=-1)
19            output = process_output.__getitem__(1).read()
20     else:
21         # Initialising empty form
22         form = HostCheckForm()
23
24     return render(request, 'host/index.html', {'form': form, 'output': output})
```

In the above code snippets, which lines have vulnerability?
**Select one alternative:**

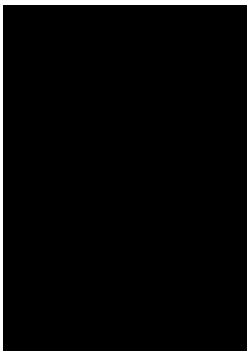
- views.py:16-19
- index.html:26-28
- views.py:14-14
- forms.py:5-6

[redacted]

## 24 CVSS (1 point)

What does CVSS stand for in the context of cybersecurity?
**Select one alternative:**

- Cyber Vulnerability Secret Service
- Critical Vulnerability Scoring System
- Common Vulnerability Security System
- Common Vulnerability Scoring System
- Cybersecurity Vulnerability Severity Scale



## 25 Another OWASP vulnerability in the code (1 point)

index.html

```
1 <!DOCTYPE html>
2 <html>
3     <head>
4         <title>Random App</title>
5     </head>
6
7     <body>
8    {# Message notifications #}
9    {% if messages %}
10       <ul class="messages">
11              {% for message in messages %}
12                 <li>{% if message.tags %} class="{{ message.tags }}"{% endif %}>{{ message }}</li>
13              {% endfor %}
14       </ul>
15    {% endif %}
16    {# Message notifications end #}
17    <h4>Team Collaborator Calendar</h4>
18         <form  method="POST" action="{% url 'teams:index' %}">
19             {% csrf_token %}
20              Email : {{form.email}}
21        {{ form.email.errors }}
22        Scheduled Task : {{form.event}}
23        {{ form.event.errors }}
24       Date : {{form.date}}
25        {{ form.date.errors }}
26
27
28            <br />
29             <input type="submit" value="submit" />
30         </form><br>
31    <hr>
32    <br>
33    {% if calendar_events %}
34       <h4>Latest events</h4>
35       {% for event in calendar_events %}
36       <b>Email: </b> {{ event.email }}<br>
37       <b>Task:</b> {{ event.event }}<br>
38       <b>Date:</b> {{ event.date }}<br><br>
39       {% endfor %}
40    {% endif%}
41
42    <br>
43 <script src="//code.jquery.com/jquery-1.10.2.js"></script>
44 <script src="//code.jquery.com/ui/1.11.4/jquery-ui.js"></script>
45 <script>
46   $(function() {
47     $( ".datepicker" ).datepicker({
48       changeMonth: true,
49       changeYear: true,
50       yearRange: "1900:2012",
51
52     });
53   });
54   </script>
55     </body>
56 </html>
```

models.py

```
1 from __future__ import unicode_literals
2
3 from django.db import models
4
5
6 #  Model for Calendar App
7 class Calendar(models.Model):
8     email = models.EmailField()
9     date = models.DateField()
10    event = models.CharField(max_length=1024)
```

views.py

```
1 from django.core.urlresolvers import reverse_lazy
2 from django.shortcuts import render
3 from django.utils.html import mark_safe
4 from django.views.generic import CreateView, TemplateView
5 # mark_safe tells django templates that a string should be used AS IS
6 from teams.forms import CalendarForm
7 from teams.models import Calendar
8
9
10 # View for scheduling task form and render scheduled task
11 class Index(CreateView):
```

```
12     form_class = CalendarForm
13     model = Calendar
14     template_name = 'teams/index.html'
15     success_url = reverse_lazy('teams:success')
16
17     # Custom function
18     def get_all_events(self):
19         temp_calendar_events = []
20         for events in Calendar.objects.all().order_by('-date'):
21             events.event = mark_safe(events.event)
22             temp_calendar_events.append(events)
23         return temp_calendar_events
24
25     # Method for form/POST data
26    def get_context_data(self, **kwargs):
27         context = super(Index, self).get_context_data(**kwargs)
28         final_events = self.get_all_events()
29         context['calendar_events'] = final_events
30         return context
31
32
33 # View for redirection
34 class Success(TemplateView):
35     template_name = 'teams/success.html'
```

In the above code snippets, which lines are vulnerable?

**Select one alternative:**

- views.py:28-28
- views.py:20-22
- index.html:22-23
- model.py:10-10

[redacted]

## 26 Public key cryptography algorithms (1 point)

Which statements regarding public key cryptography algorithm are FALSE?

1. Message sender and receiver use identical keys when they use public key cryptography algorithms.
2. The public key cryptography algorithms are usually open to public.
3. Stream cipher is a public key cryptography algorithm.
4. ECDSA is not a public key cryptography algorithm.

**Select one alternative:**

- 2, 3, and 4
- 1, 3, and 4
- All of them
- 1, 2, and 4

[redacted]

## 27 Buffer overflow (1 point)

Which of these kinds of inputs can cause a buffer overflow?
1. An environment variable
2. String input from the user
3. A single integer
4. A floating point number
5. File input

**Select one alternative:**

- All of the above
- 3 and 4
- 1 and 2
- 2 and 5

[redacted]

## 28 Security requirements (1 point)

Which of these is a good security requirement?
**Select one alternative:**

- The system should be free from vulnerabilities
- The system must have good usability
- End user data should be encrypted at rest
- The system shall work just like the previous one, but on a new platform
- The system shall encrypt all confidential data using the RSA algorithm

[redacted]

## 29 Cookies (1 point)

What is the security issue of cookie-based tokens?
**Select one alternative:**

- Web browser can not save the cookie value
- Web browser can not see the cookie expiration time
- Server does not see which domain sends the cookie
- Cookies are unhealthy for the end user
- Server can not save the cookie value

[redacted]

## 30 Static code analysis (1 point)

Which approach does NOT belong to static code analysis for vulnerability detection?
**Select one alternative:**

- Control flow analysis
- Pattern matching
- Taint analysis
- Penetration testing

[redacted]

## 31 Supply chain vulnerability countermeasures (1 point)

Which is NOT a transparency countermeasure of software supply chain security?
**Select one alternative:**

- Version Locking
- Software Bill of Materials (SBOM)
- Sigstore
- Dependabot

[redacted]

## 32 Microservice architecture security countermeasures (1 point)

What is the countermeasure to defend against attacks targeting the load balancer in the microservice architecture?
**Select one alternative:**

- Service-level authorization
- Service-to-service authentication
- Rate throttling
- Secure container

[redacted]

# Case description: Risk assessment of a risk assessment tool for Air Traffic Management (ATM)

SESAR Joint Undertaking defines, develops and deploys technologies to transform air traffic management (ATM) in Europe. These technologies are known as *solutions* and are developed in numerous projects under det SESAR JU programme. Solutions can be used to manage conventional aircrafts, drones, air taxis and vehicles flying at higher altitudes, and need to undergo risk assessments at various stages of their development lifecycle (i.e. concept development, lab experiments, prototypes in realistic environments, proven system in operation development). One of the solutions is a cyber security risk assessment methodology, that is to be applied to the other solutions in order to derive their security requirements and maintain an up-to-date risk picture. A part of this solution is a web-based tool that is intended to make risk assessments easier for other air traffic management solutions. This includes defining scope and goals, identifying assets, threats and vulnerabilities, describing and evaluating risks and deriving security requirements. Since this web-based tool is considered to be a solution by itself, you will also need to perform a risk assessment of it (effectively a risk assessment of a risk assessment tool).

You have been given the following business goals:

- BG1: Support the ATM risk assessment methodology.
- BG2: A user-friendly web-interface that supports various stakeholders involved.
- BG3: Able to retrieve assets from a digital catalogue of reusable items (e.g. flight data, satellite datalink, primary radar, air traffic controller, passengers.).
- BG4: Preserve confidentiality of the assessments of the SESAR solutions.
- BG5: Allow sharing of risk assessment information between authorized people (involved in an assessment).

## Part 1 tasks (30 points in total)

In this part you will perform tasks related web-based tool from the case description.

If you feel that any of the tasks require information that you do not find in the text, then you should:

- Document the necessary assumptions (e.g. technology, standards, software, design choices.)
- Explain why you need them.

Your answers should be brief and to the point.

Task 1: As part of defining the scope, list at least five impact dimensions you consider relevant for this assessment. (3 points)

[redacted]

Task 2: What kind of people/stakeholders would you involve in the assessment? Explain why. (3 points)

[redacted]

Task 3: What kind of access control model would you recommend for this solution? (2 points)

[redacted]

Task 4: Identify 5 assets (something of value that needs protection) related to the tool. (3 points)

[redacted]

Task 5: In the use case diagram in the next page, you can see use cases and undefined actors. Define at least 5 suitable actors and describe how they should be connected referring to the use case labels (you can add more actors if needed). (3 points)

[redacted]

Task 6: Define at least 5 corresponding misuse case elements and describe how they should be connected to the use cases (you use the labels as a references). (3 points)

[redacted]

Task 7: The last page shows a DFD. Explain the different types of elements in the diagram. Identify possible attack points in relation to the DFD elements and describe at least 5 threats according to STRIDE. (4 points)

[redacted]

[redacted]

Task 8: Based on the threat models, identify at least 5 technical risks and evaluate them. You should describe necessary assumption related to the technology. (3 points)

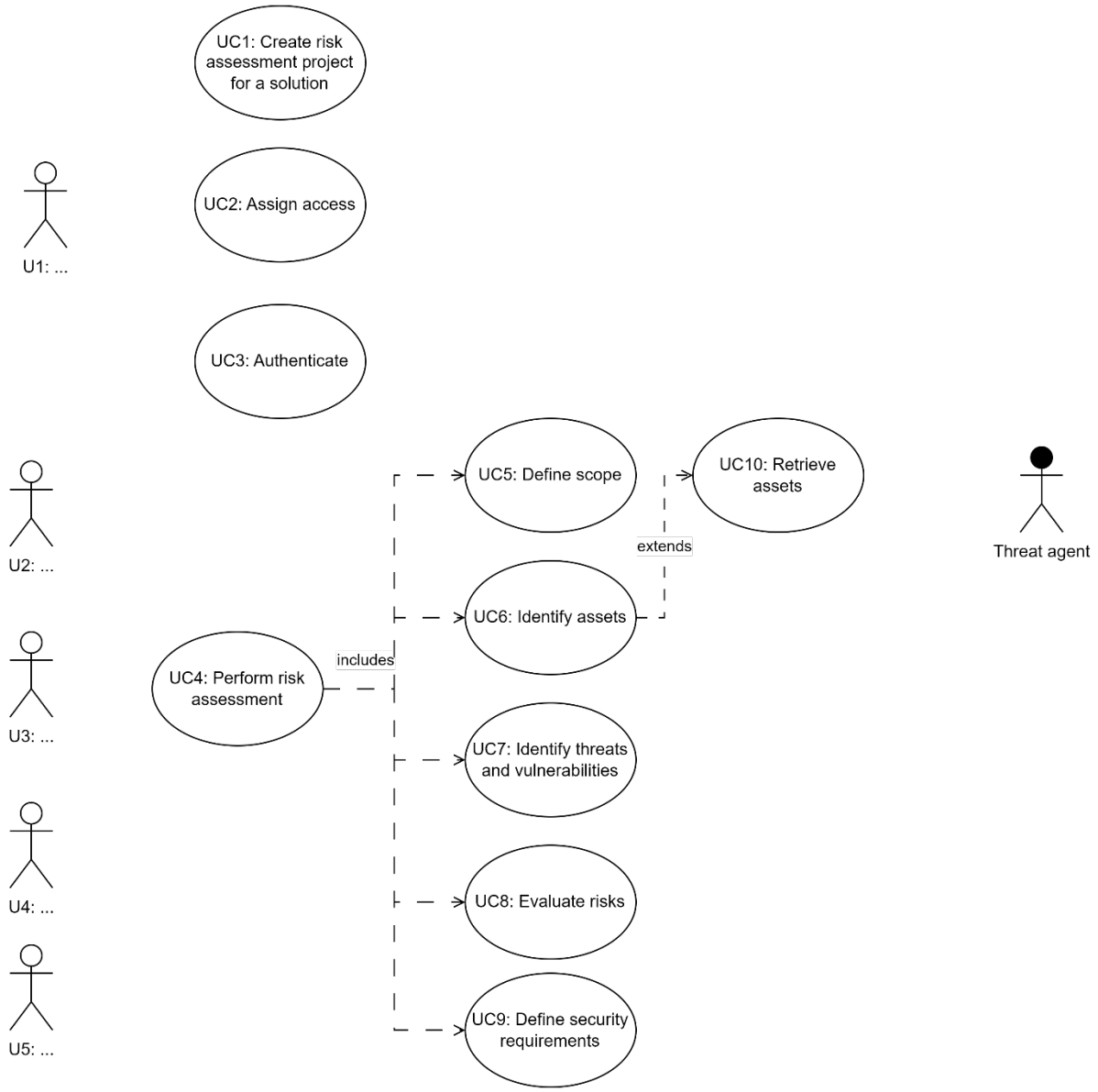
[redacted]

Task 9: Select at least 3 technical risks and define one well-formulated security requirement for each. (3 points)

[redacted]

Task 10: Write a short reflection on which threat agents you consider to be significant for this tool. Justify these using principles from attacker-centric threat modelling. (3 points)

[redacted]


UC1: Create risk assessment project for a solution
UC2: Assign access
UC3: Authenticate
U1: ...
U2: ...
U3: ...
U4: ...
U5: ...
UC4: Perform risk assessment
includes
UC5: Define scope
UC6: Identify assets
UC7: Identify threats and vulnerabilities
UC8: Evaluate risks
UC9: Define security requirements
extends
UC10: Retrieve assets
Threat agent